KIẾN TRÚC OOP VÀ LUỒNG XỬ LÝ POC ĐÁNH GIÁ CÂU TRẢ LỜI

Mục tiêu: đánh giá phần thể hiện trong một câu trả lời dựa trên các tiêu chí và thang điểm đã khai báo; trả phản hồi gắn với căn cứ. Đầu vào là Question, các Rubric và Answer.originalText. Đầu ra là EvaluationReport chứa kết quả từng rubric, bằng chứng, lý do và Feedback.

Tài liệu mô tả mã OOP hiện tại, từ khởi chạy đến lưu kết quả. Tên lớp, thuộc tính, phương thức và thông báo trong code dùng tiếng Anh. Phần giải thích dùng tiếng Việt; không đưa kết quả chạy thử vào báo cáo.

1. Vì sao chuyển sang OOP và tổ chức thế nào?

Các thành phần có trách nhiệm rõ được đóng gói thành đối tượng, liên kết qua constructor. EvaluationService điều phối; RubricRules chấm theo thang; EvidenceReview kiểm tra căn cứ; FeedbackBuilder tạo phản hồi. Điều này giúp theo dõi phụ thuộc, đặt điểm dừng debug và thay từng thành phần mà không sao chép quy tắc chấm.

Vị trí	Trách nhiệm trong MVC-S
source/main.py	Application tạo Repository, các thành phần Service, Controller và View. main() chỉ là hàm khởi chạy.
source/Model/	Các lớp dữ liệu, kiểm tra trường và snapshot. repositories/ chứa hợp đồng persistence và cách cài JSON.
source/service/	Các lớp nghiệp vụ: xác thực rubric, diễn giải, kiểm tra bằng chứng, tính điểm, phản hồi và điều phối.
source/Control/	EvaluationController xác thực request và chuyển lỗi dự kiến thành EvaluationResponse.
source/view/	EvaluationView nhận tham số, gọi Controller và trình bày kết quả; không đọc file dữ liệu hoặc gọi AI.
source/reports/	Báo cáo PDF đứng cùng cấp các thư mục kiến trúc. results/ là đầu ra JSON phát sinh, không được đưa vào báo cáo này.
test/; examples/evaluateAnswer/	test/ chứa kiểm thử, ví dụ nằm trong examples/. Không có chương trình đánh giá thứ hai trong các thư mục này.

Theo AGENTS.md, nghiệp vụ thuộc **service**, cấu trúc dữ liệu và persistence thuộc **Model**. Ngoài main.py và dấu mốc package __init__.py, source không có file code đứng ngang các thư mục. Các module có chữ evaluation mô tả những trách nhiệm khác nhau, không phải nhiều thuật toán chấm.

Tất cả logic ứng dụng của PoC nằm trong các phương thức. Các lớp dữ liệu tiếp tục dùng Pydantic; helper kiểm thử vẫn là hàm pytest. Tên do Python/Pydantic quy định như __init__, model_config, model_validate giữ đúng API thư viện. Tên do ứng dụng đặt dùng camelCase; tên lớp dùng PascalCase.

2. Quan hệ giữa các lớp chính

Dấu --> dưới đây có nghĩa là giữ tham chiếu hoặc gọi đối tượng; không biểu diễn kế thừa. Các phụ thuộc được tạo tại Application và truyền vào constructor.

```
Application
  --> EvaluationView
        --> EvaluationController
              --> EvaluationService
                    --> EvaluationRepository [Protocol]
                    --> InterpreterFactory
                    --> RubricRules
                    --> EvidenceReview
                    --> FeedbackBuilder

EvaluationRepository <.. JsonEvaluationRepository
EvidenceInterpreter  <.. GeminiInterpreter
EvidenceInterpreter  <.. RecordedInterpreter

InterpreterFactory --> creates GeminiInterpreter / RecordedInterpreter
EvaluationService  --> uses EvidenceInterpreter.interpret(inputs)
GeminiInterpreter  --> EvaluationPrompt
RecordedInterpreter --> RecordedReview
```

Dấu <.. chỉ việc đáp ứng Protocol theo cấu trúc: có các thuộc tính và phương thức yêu cầu. Các implementation không khai báo kế thừa trực tiếp Protocol; Python vẫn cho phép dùng qua cùng hợp đồng.

Quan hệ	Ý nghĩa cụ thể
Application - View - Controller - Service	Mỗi đối tượng giữ tham chiếu tới lớp tiếp theo. View gọi submitAnswer; Controller gọi evaluateFiles. Không có quan hệ cha-con bằng kế thừa giữa các tầng.
Service - các phụ thuộc	Liên kết sử dụng: Service nhận Repository, Factory, Rules, Review và FeedbackBuilder. Các đối tượng được truyền vào có thể được dùng lại; không giả định chúng bị hủy cùng Service.
Service - EvidenceInterpreter	Phụ thuộc theo từng lượt: interpreter được chọn rồi truyền vào assessAnswer/evaluateCriteria/readReport; không giữ kết quả của lượt trước trong Service.
FrozenData - các model PoC	Kế thừa thật: model đầu vào/kết quả kế thừa FrozenData, lớp này kế thừa Pydantic BaseModel để dùng cùng cấu hình validation và frozen.
EvaluationInput - Question/Rubric/Answer	Quan hệ chứa dữ liệu: một câu hỏi, từ 1 đến 8 rubric theo năng lực và một bài gốc. Rubric chứa tiêu chí và mức điểm.
Result - Evidence - Feedback	Mỗi rubric có một result; mỗi criterion có một check chứa evidence. Feedback tham chiếu resultId. Các ID được kiểm tra với input gốc.

AnswerEvaluation giữ các snapshot: chưa chấm -> có results -> có feedback. Mỗi thao tác trả snapshot mới; không sửa bài gốc. Một instance EvaluationService có thể xử lý nhiều lượt liên tiếp mà không dùng chung results của các lượt.

3. Mẫu thiết kế và kỹ thuật thực sự được dùng

Mẫu / kỹ thuật	Áp dụng và lợi ích
Strategy	EvidenceInterpreter là hợp đồng interpret(inputs). GeminiInterpreter và RecordedInterpreter là hai cách cung cấp diễn giải có thể thay thế. Service gọi cùng giao diện; thuật toán tính điểm giữ nguyên.
Adapter	GeminiInterpreter chuyển dữ liệu nội bộ thành HTTP/JSON của Gemini rồi chuyển phản hồi thành Interpretation. Chi tiết mạng, candidate và lỗi provider được cô lập tại adapter.
Repository	EvaluationRepository khai báo đọc/lưu. JsonEvaluationRepository cài thao tác file; Service dùng hợp đồng đó. Có thể thay bằng kho khác đáp ứng đủ hợp đồng và không ghi đè kết quả.
Simple Factory	InterpreterFactory.createInterpreter chọn và tạo provider theo cấu hình. Đây là factory đơn giản; không dùng cơ chế lớp con ghi đè tạo đối tượng của mẫu GoF Factory Method.
Dependency Injection	Application truyền các đối tượng qua constructor. Đây là kỹ thuật quản lý phụ thuộc: nhìn vào constructor sẽ biết lớp cần gì; có thể truyền bản giả lập khi kiểm thử.
MVC-S / Service Layer	Là cách tổ chức kiến trúc: View nhận/hiển thị, Controller xử lý request, Service nghiệp vụ, Model dữ liệu/persistence. Không đồng nghĩa với kế thừa giữa các lớp.

FeedbackBuilder chỉ là tên lớp tạo phản hồi bằng một phương thức; không triển khai GoF Builder. Các thao tác createEvaluation/evaluateCriteria/generateFeedback là quy trình có kiểm tra trạng thái, chưa phải mẫu State với các lớp trạng thái riêng. Không cần thêm hệ thống kế thừa để đặt tên nhiều pattern.

4. Luồng chạy từ khi nhận câu trả lời

Khởi tạo: main(arguments) -> Application.__init__ tạo các phụ thuộc -> Application.run(arguments) -> EvaluationView.run(arguments). View.readArguments chuyển tham số CLI thành Namespace; View tạo requestData gửi Controller.

Bước và lớp chủ trì	Lời gọi tiếp theo / dữ liệu trả về
1. Controller.submitAnswer	EvaluationRequest.model_validate -> checkRecording. Gọi Service.evaluateFiles với các đường dẫn và tên provider; dữ liệu sai dừng tại biên request.
2. Service.evaluateFiles	Repository.resolveOutput -> Service.loadInputs -> Repository.loadInput/loadAnswer. Nếu recorded: loadRecording. Factory.createInterpreter tạo nguồn diễn giải.
3. Service.assessAnswer	createEvaluation -> evaluateCriteria -> generateFeedback -> readReport. Đây là quy trình trong bộ nhớ; các bước đọc/lưu nằm ở evaluateFiles.
4. Diễn giải và chấm	RubricRules.validateInputs trước khi gọi interpreter.interpret. EvidenceReview.validateInterpretation chuẩn hóa căn cứ. RubricRules.scoreChecks tính mức cho mỗi rubric; tạo EvaluationResult.
5. Phản hồi và kết quả	Service.validateResults kiểm tra tính nhất quán. FeedbackBuilder.buildFeedback trả từng phản hồi. readReport thêm inputHash và metadata, tạo EvaluationReport.
6. Lưu và trả về	Repository.saveReport lưu báo cáo hoàn chỉnh. Controller tạo EvaluationResponse -> checkOutcome. View in text qua renderReport hoặc JSON; in đường dẫn và trả mã thoát.

4.1. Cây gọi phương thức chi tiết

Tên rút gọn bên trái trỏ tới các instance trong phần quan hệ lớp. Cây ghi đúng trình tự thực thi; nhánh tùy chọn đặt trong ngoặc vuông.

```
EvaluationView.run(arguments)
  readArguments(arguments)
  controller.submitAnswer(requestData)
    EvaluationRequest.model_validate() -> checkRecording()
    service.evaluateFiles(...)
      repository.resolveOutput(outputPath)
      loadInputs(inputPath, answerPath)
        repository.loadInput() -> readTextFile()
        [new answer] repository.loadAnswer() -> readTextFile()
        [new answer] EvaluationInput.model_validate(...)
      [recorded] repository.loadRecording() -> readTextFile()
      interpreterFactory.createInterpreter(providerName, recording)
        recorded -> RecordedInterpreter(recording)
        gemini -> GeminiSettings.fromEnvironment()
               -> GeminiInterpreter(...)
      assessAnswer(inputs, interpreter)
        createEvaluation(...) -> rubricRules.validateInputs()
          requireUnique(); validateLevels() -> findLevel()
        evaluateCriteria(evaluation, interpreter)
          rubricRules.validateInputs()
          interpreter.interpret(inputs)
          evidenceReview.validateInterpretation(...)
            validateCheck() -> validateQuote() / validateOmission()
          [each rubric] rubricRules.scoreChecks() -> findLevel() x2
          EvaluationResult(...) -> checkScore()
        generateFeedback(evaluation)
          validateResults(evaluation)
          [each result] feedbackBuilder.buildFeedback(...)
        readReport(evaluation, interpreter)
          validateResults(evaluation)
          inputs.inputDigest(); EvaluationReport(...)
      repository.saveReport(report, resultPath)
    EvaluationResponse(report, outputPath) -> checkOutcome()
  renderReport(report) OR report.model_dump_json()
  print report; print saved path; return exit code
```

validateResults: gọi rubricRules.validateInputs; dựng các RubricReview từ results rồi gọi evidenceReview.validateInterpretation; kiểm tra answerId; gọi scoreChecks để đối chiếu score/status/reason; nếu có feedback thì gọi buildFeedback để đối chiếu. Đây là việc kiểm tra lại tại ranh giới thao tác, không phải các bản cài thuật toán khác nhau và không gọi AI thêm lần nữa.

4.2. Trích xuất thông tin và nhận diện mơ hồ

Nhánh Gemini: GeminiInterpreter.interpret -> buildRequest -> EvaluationPrompt.systemPrompt và promptPayload -> HTTP request -> parseResponse -> Interpretation. Prompt hướng dẫn chọn phần liên quan từng criterion, chấp nhận lời giải tương đương và nhận diện phần mơ hồ; AI không được trả điểm. Prompt chỉ gửi tiêu chí và bài gốc, không gửi scoreLevels.

Nhánh recorded: RecordedInterpreter.interpret -> inputs.inputDigest -> so inputHash -> trả Interpretation đã ghi sẵn. Nhánh này minh họa luồng xử lý, không phân tích một bài mới.

Làm rõ: nội dung mơ hồ được ghi ambiguity và insufficientEvidence, kèm observation nêu điều chưa rõ. scoreChecks chỉ ra criterion cần làm rõ khi nó có thể đổi mức. PoC chưa có vòng hội thoại hỏi lại, không tự suy diễn bổ sung nội dung thiếu.

5. Quy tắc chấm, bất biến và xử lý lỗi

Quy tắc	Cách thực hiện / lý do
Thang điểm nhiều mức	Rubric.scoreLevels giữ value, description, condition, requiredMet và minimumMet; độ dài danh sách là số mức. findLevel chọn mức cao nhất thỏa điều kiện máy đọc. Không suy ngưỡng từ mô tả bằng văn bản.
Hai biên khi chưa đủ căn cứ	scoreChecks tính mức từ met và mức từ met + insufficientEvidence. Hai mức khác nhau: score=null. Hai mức giống nhau: trả mức đó và vẫn lưu tiêu chí chưa rõ. Các điều kiện dương bảo đảm thêm met không làm giảm mức.
Sai khác với chưa rõ	unmet có căn cứ sai hoặc thiếu nội dung bắt buộc mà omissionMeansUnmet cho phép. insufficientEvidence không bị đổi tự động thành 0 hoặc identifiedIssues.
Bài gốc và bằng chứng	Dữ liệu frozen, thao tác trả snapshot mới. Trích dẫn phải đúng nguyên văn và [startOffset:endOffset]. Quote lặp cần offsets rõ. Omission không giả lập trích dẫn.
Liên kết và tính nhất quán	Một kết quả cho mỗi rubric, một check mỗi criterion, một feedback mỗi result trong luồng hoàn chỉnh. ID, điểm, reason và feedback được validateResults kiểm tra trước khi xuất.
Ngữ nghĩa còn cần xem xét	Kiểm tra Python xác nhận trích dẫn tồn tại, liên kết và phép tính đúng; chưa tự chứng minh nhận định ngữ nghĩa của AI đúng. Cần bộ câu hỏi và nhận định chuẩn được con người duyệt để đánh giá chất lượng AI.
Lỗi xử lý	Lỗi request/schema/rubric/bằng chứng/provider/lưu trữ dừng quy trình. Controller chuyển lỗi dự kiến thành EvaluationFailure. Không tạo điểm dự phòng; không bắt mọi lỗi lập trình làm mất traceback.
Lưu kết quả	JSON được ghi hoàn chỉnh vào file tạm cùng thư mục, flush/fsync, tạo đích bằng hard link rồi dọn file tạm. Đích đã tồn tại thì lỗi; đầu vào không bị ghi đè. Cần filesystem hỗ trợ hard link.

Ví dụ hai biên: rubric có mức 0 khi chưa đạt tiêu chí nào, mức 1 khi đạt A, mức 2 khi đạt A và B. Nếu A=met, B=insufficientEvidence, điểm có thể là 1 hoặc 2 nên trả null. Nếu A=unmet và B chưa rõ, B không thể làm thỏa A; mức vẫn là 0, nhưng B vẫn mang trạng thái chưa rõ.

Các bất biến trên là điều kiện phải được bảo toàn qua thao tác hợp lệ, không có nghĩa mọi tình huống đều chạy cùng một nhánh. Request sai hoặc lỗi AI có thể làm thao tác thất bại; khi đó không công bố một kết quả chấm dở dang.

6. Danh mục file, lớp và phương thức

Các đường dẫn từ đây tính tương đối từ source/. Với lớp chỉ khai báo dữ liệu, bảng mô tả lớp thay vì liệt kê những constructor do thư viện tự sinh.

main.py

Application là composition root: tạo và nối các đối tượng cụ thể. main() chỉ khởi chạy, không chứa nghiệp vụ.

Hàm / lớp	Nhiệm vụ, đầu vào và đầu ra
Application.__init__(reportDirectory)	Chọn thư mục kết quả; tạo JsonEvaluationRepository, InterpreterFactory, RubricRules, EvidenceReview, FeedbackBuilder; truyền chúng vào EvaluationService; tạo Controller rồi View và giữ self.view.
Application.run(arguments)	Gọi self.view.run(arguments) và trả mã thoát.
main(arguments)	Tạo Application, gọi run; khối __main__ chuyển giá trị trả về thành mã thoát hệ điều hành.

Control/evaluationSchemas.py

Hợp đồng request/response tại ranh giới giao diện. Dữ liệu đầu vào không hợp lệ bị chặn trước khi chạy Service.

Hàm / lớp	Nhiệm vụ, đầu vào và đầu ra
EvaluationRequest; checkRecording()	Giữ inputPath, providerName và các đường dẫn tùy chọn. checkRecording yêu cầu recordingPath khi và chỉ khi provider là recorded; trả chính request đã hợp lệ.
EvaluationFailure	Dữ liệu lỗi gồm code, message, retryable và fields. Không chứa điểm hoặc kết luận về người học.
EvaluationResponse; checkOutcome()	Chứa report + outputPath hoặc chỉ error. checkOutcome loại trạng thái thiếu một nửa kết quả hoặc vừa thành công vừa lỗi.

Control/evaluationController.py

Nhận hành động từ View, điều phối Service và chuẩn hóa lỗi dự kiến; không tự đọc file hoặc chấm điểm.

Hàm / lớp	Nhiệm vụ, đầu vào và đầu ra
EvaluationController.__init__(service)	Nhận Service qua tham số để có thể thay bằng đối tượng khác khi tích hợp hoặc kiểm tra.
submitAnswer(requestData)	Xác thực EvaluationRequest, gọi evaluateFiles và trả EvaluationResponse. Chuyển EvaluationError, ValidationError, FileExistsError, UnicodeError và OSError thành các mã lỗi tương ứng. Không bắt mọi ValueError.

view/evaluationView.py

Chứa toàn bộ giao diện terminal của PoC. Việc in không làm thay đổi dữ liệu hay tính điểm.

Hàm / lớp	Nhiệm vụ, đầu vào và đầu ra
EvaluationView.__init__(controller)	Giữ Controller được truyền vào; View không tự tạo Service, Repository hoặc provider.
readArguments(arguments)	Nhận --input, --provider, --recording, --answerFile, --output, --format và trả argparse.Namespace.
run(arguments)	Gọi readArguments, cấu hình UTF-8 cho terminal rồi gọi self.controller.submitAnswer một lần, in lỗi ra stderr với exit code 1; nếu thành công, in text/JSON và đường dẫn lưu, trả 0.
renderReport(report)	Tạo chuỗi trình bày nguồn diễn giải, phạm vi, mức đạt, mức cho phép, feedback và trích dẫn. Nhận báo cáo hoàn chỉnh từ luồng đánh giá.

Model/answerData.py

Schema đầu vào dùng chung. Pydantic kiểm tra kiểu, giới hạn và cấm trường thừa; quy tắc liên kết nhiều đối tượng nằm ở Service.

Hàm / lớp	Nhiệm vụ, đầu vào và đầu ra
FrozenData	Lớp nền bất biến, extra=forbid và validate_default=True.
Question; Answer	Question giữ ID/version, đề, yêu cầu, ngữ cảnh và mức dự kiến. Answer giữ ID/revision, questionId và nguyên văn; không tự chuẩn hóa hoặc cắt nội dung trả lời.
Criterion	Mô tả tiêu chí, acceptanceRule, reviewRule và omissionMeansUnmet. Các trường này hướng dẫn diễn giải bằng chứng.
ScoreLevel; Rubric	ScoreLevel giữ điểm, mô tả, điều kiện và ngưỡng máy đọc. Rubric nhóm tiêu chí và thang điểm theo một năng lực, tham chiếu câu hỏi.
EvaluationInput	Gói Question, tuple Rubric, Answer và cờ exampleOnly thành snapshot của lượt đánh giá.
EvaluationInput.inputDigest()	Chuẩn hóa JSON với key được sắp xếp rồi tính SHA-256. Thay câu hỏi, rubric, bài hoặc metadata đầu vào sẽ đổi hash; thuật toán và JSON giữ tương thích recording cũ.
Text, Score, Count, Version	Kiểu dữ liệu có giới hạn: văn bản có nội dung, điểm hữu hạn, đếm tiêu chí có giới hạn, phiên bản nguyên dương.

Model/evaluationData.py

Schema diễn giải, trạng thái và kết quả. Các trạng thái chưa đủ căn cứ được biểu diễn rõ để không lẫn với điểm 0.

Hàm / lớp	Nhiệm vụ, đầu vào và đầu ra
Evidence; CriterionCheck	Evidence giữ loại, quote, vị trí, observation và offsets. CriterionCheck giữ criterionId, status và một hay nhiều Evidence.
RubricReview; Interpretation	Gom các check theo rubric và gom tất cả review thành đầu ra của interpreter.
EvaluationResult; checkScore()	Kết quả một rubric với resultId, answerId, score, status, checks và reason. checkScore yêu cầu graded có điểm và insufficientEvidence có score=null.
Feedback; AnswerEvaluation	Feedback liên kết resultId với giải thích và các điểm chưa đạt. AnswerEvaluation giữ snapshot đầu vào cùng results/feedbackItems trung gian.
EvaluationReport; RecordedReview	Report giữ inputs, inputHash, provider/model/prompt và kết quả. RecordedReview giữ inputHash với interpretation để phát lại đúng dữ liệu.

service/evaluationService.py

Một file điều phối toàn bộ đánh giá. Lớp EvaluationService quản lý các bước xử lý và dùng những đối tượng chuyên trách; trạng thái từng lượt nằm trong AnswerEvaluation, không giữ trong self.

Hàm / lớp	Nhiệm vụ, đầu vào và đầu ra
EvaluationService.__init__(repository, interpreterFactory, rubricRules, evidenceReview, feedbackBuilder)	Giữ năm đối tượng được truyền vào. repository tuân theo EvaluationRepository; nguồn diễn giải tuân theo EvidenceInterpreter. Không tự tạo các quy tắc hoặc kho lưu trong Service.
loadInputs(inputPath, answerPath)	Đọc EvaluationInput. Nếu có answerPath, giữ nguyên văn file mới, tạo answerId mới/revision=1 và xác thực lại gói dữ liệu. Trả EvaluationInput mới; không ghi đè đầu vào.
evaluateFiles(...)	Kiểm tra lựa chọn provider, chọn nơi lưu, đọc input/recording, tạo interpreter, gọi assessAnswer, lưu báo cáo hoàn chỉnh và trả (report, resultPath).
assessAnswer(inputs, interpreter)	Phương thức quy trình trong bộ nhớ: createEvaluation -> evaluateCriteria -> generateFeedback -> readReport. Không đọc/ghi file.
createEvaluation(question, rubrics, answer, exampleOnly)	Tạo EvaluationInput, gọi validateInputs, trả AnswerEvaluation chưa có results/feedback.
evaluateCriteria(evaluation, interpreter)	Chỉ nhận trạng thái chưa chấm. Lấy diễn giải, xác minh, gọi scoreChecks cho từng rubric, tạo EvaluationResult và trả bản trạng thái mới.
generateFeedback(evaluation)	Yêu cầu đã có results và chưa có feedback. Gọi validateResults rồi buildFeedback cho mỗi result; trả trạng thái mới.
validateResults(evaluation)	Kiểm tra ID duy nhất, đủ review/check, answerId đúng, điểm/lý do khớp scoreChecks và feedback khớp buildFeedback. Không trả kết quả; báo lỗi nếu sai.
readReport(evaluation, interpreter)	Kiểm tra trạng thái, thêm inputHash và metadata provider, trả EvaluationReport. ADT có thể đọc trạng thái mới tạo; luồng evaluateFiles chỉ lưu sau khi đã hoàn tất feedback.

service/rubricRules.py

Lớp RubricRules là nơi duy nhất định nghĩa cách xác thực rubric và ánh xạ các criterionCheck sang mức điểm.

Hàm / lớp	Nhiệm vụ, đầu vào và đầu ra
requireUnique(values, label)	So số phần tử với tập hợp; phát hiện trùng rubricId, criterionId, competency hoặc requiredMet.
validateInputs(inputs)	Kiểm tra answer và rubric thuộc đúng question, ID/năng lực không lặp; gọi validateLevels cho mỗi rubric.
validateLevels(rubric)	Yêu cầu điểm khác nhau tăng dần, mức nền không đòi tiêu chí, tham chiếu/ngưỡng hợp lệ. Duyệt các tập tiêu chí để bảo đảm từng mức có thể đạt; số tiêu chí tối đa là 12.
findLevel(rubric, metIds)	Duyệt mức từ cao xuống thấp; trả mức đầu tiên có requiredMet là tập con của metIds và đủ minimumMet.
scoreChecks(rubric, checks)	Tính hai biên bằng findLevel. Trả (score, reason); score=null nếu phần chưa rõ có thể thay đổi mức. Không cộng điểm các năng lực.

service/evidenceReview.py

Lớp EvidenceReview kiểm tra hình dạng và liên kết của bằng chứng trước khi dùng cho điểm; giữ căn cứ có thể truy về nguyên văn.

Hàm / lớp	Nhiệm vụ, đầu vào và đầu ra
invalidEvidence(message)	Phát sinh EvaluationError có code invalidEvidence để các nhánh xác thực dùng cùng cơ chế lỗi.
validateQuote(evidence, originalText)	Kiểm tra quote nguyên văn, đủ hai offsets, khoảng vị trí hợp lệ. Nếu quote duy nhất và thiếu cả hai offsets, tự xác định vị trí. Trả Evidence có location chuẩn hóa.
validateOmission(evidence, criterion, check)	Yêu cầu quote/offsets null, không dùng omission để chứng minh met, chỉ kết luận unmet khi rubric cho phép. Trả bằng chứng ghi phạm vi toàn bài.
validateCheck(check, criterion, originalText)	Kiểm tra từng Evidence, buộc ambiguity đi với insufficientEvidence và met phải có excerpt. Trả check đã chuẩn hóa.
validateInterpretation(inputs, interpretation)	Buộc đúng một review mỗi rubric, một check mỗi criterion, không ID lạ/trùng/thiếu. Gọi validateCheck và sắp theo thứ tự rubric đầu vào.

service/feedbackBuilder.py

Lớp FeedbackBuilder tạo nội dung phản hồi từ kết quả đã xác minh để điểm và giải thích có cùng căn cứ.

Hàm / lớp	Nhiệm vụ, đầu vào và đầu ra
buildFeedback(result, rubric)	Ghép reason và observations theo từng criterion; tạo identifiedIssues cho unmet với acceptanceRule tương ứng. Trả Feedback. Không thêm một lần chấm AI.

service/evaluationErrors.py

Kiểu lỗi nghiệp vụ dùng giữa Service và Controller; giúp lỗi xử lý không biến thành kết luận về bài làm.

Hàm / lớp	Nhiệm vụ, đầu vào và đầu ra
EvaluationError.__init__(code, message, retryable)	Giữ mã lỗi, nội dung tiếng Anh và cờ có thể thử lại. Các chỗ bắt lỗi mạng dùng raise ... from ... để giữ nguyên nhân khi debug.

service/providers/evidenceInterpreter.py

EvidenceInterpreter định nghĩa giao diện nguồn diễn giải. Hai provider đáp ứng Protocol theo cấu trúc, không cần kế thừa nó.

Hàm / lớp	Nhiệm vụ, đầu vào và đầu ra
EvidenceInterpreter.interpret(inputs)	Protocol nhận EvaluationInput và trả Interpretation; đồng thời cung cấp providerName, modelName, promptVersion. Không tự cài việc chấm.

service/providers/interpreterFactory.py

InterpreterFactory tập trung việc tạo nguồn diễn giải; thay thế hàm chọn provider trước đây trong main.

Hàm / lớp	Nhiệm vụ, đầu vào và đầu ra
createInterpreter(providerName, recording)	recorded với recording hợp lệ: tạo RecordedInterpreter. gemini không kèm recording: đọc GeminiSettings.fromEnvironment rồi tạo GeminiInterpreter. Cấu hình khác báo providerConfig. Trả đối tượng đáp ứng EvidenceInterpreter; chưa gọi AI.

service/providers/recordedInterpreter.py

Nguồn diễn giải có sẵn cho PoC; thay Gemini thông qua cùng interface.

Hàm / lớp	Nhiệm vụ, đầu vào và đầu ra
RecordedInterpreter.__init__(recording)	Nhận RecordedReview đã xác thực cấu trúc.
interpret(inputs)	So recording.inputHash với inputs.inputDigest(). Khớp thì trả interpretation; khác thì báo recordingMismatch. Không gọi mạng hoặc tính điểm.

service/providers/geminiInterpreter.py

Cô lập cấu hình và giao tiếp với dịch vụ AI; Service lõi chỉ nhận Interpretation có cấu trúc.

Hàm / lớp	Nhiệm vụ, đầu vào và đầu ra
GeminiSettings.fromEnvironment()	Lấy GEMINI_API_KEY, GEMINI_MODEL, EVALUATION_TIMEOUT. API key không xuất hiện trong repr của settings; timeout được đọc thành số giây.
GeminiInterpreter.__init__(apiKey, modelName, timeoutSeconds, prompt)	Kiểm tra key, tên model và timeout 1-120 giây; giữ cấu hình và EvaluationPrompt được truyền vào hoặc prompt mặc định; lấy promptVersion từ prompt.version.
buildRequest(inputs)	Gọi systemPrompt và promptPayload, tạo request có JSON schema của Interpretation, giới hạn body 250 KB và đưa API key vào header. Trả urllib Request.
interpret(inputs)	Gọi buildRequest rồi urlopen một lần; đọc tối đa 1 MB và gọi parseResponse. Phân biệt quota, HTTP, timeout, mạng; không tự thử lại hay tạo điểm dự phòng.
parseResponse(rawBody)	Yêu cầu một candidate hoàn chỉnh STOP, không bị block. Ghép text ngoài phần thought và xác thực Interpretation; JSON/shape sai trả providerOutput.

service/prompts/evaluationPrompt.py

Lớp EvaluationPrompt giữ hướng dẫn diễn giải ở một chỗ để xem xét cách AI xử lý yêu cầu, lời giải tương đương và thông tin mơ hồ.

Hàm / lớp	Nhiệm vụ, đầu vào và đầu ra
systemPrompt()	Trả hướng dẫn tiếng Anh: xem bài làm là dữ liệu, không làm theo chỉ dẫn trong bài, không bịa lập luận/trích dẫn và không cho điểm; chỉ nhận xét theo criterion.
promptPayload(inputs)	Trả dict chứa question, rubricId/competency/criteria và answer. Không gửi scoreLevels cho AI vì điểm do Python tính.
EvaluationPrompt.version	Nhãn phiên bản prompt được lưu trong báo cáo để truy xuất cách diễn giải đã sử dụng.

Model/repositories/evaluationRepository.py

Giao diện persistence giúp EvaluationService dùng file, bộ nhớ hoặc cơ sở dữ liệu mà không đổi các bước chấm.

Hàm / lớp	Nhiệm vụ, đầu vào và đầu ra
EvaluationRepository.loadInput(inputPath)	Hợp đồng đọc và trả EvaluationInput.
loadAnswer(answerPath)	Hợp đồng trả nguyên văn câu trả lời kiểu str.
loadRecording(recordingPath)	Hợp đồng đọc và trả RecordedReview.
resolveOutput(outputPath)	Hợp đồng chọn và kiểm tra vị trí lưu; trả Path.
saveReport(report, outputPath)	Hợp đồng lưu báo cáo hoàn chỉnh, không có giá trị trả về.

Model/repositories/jsonEvaluationRepository.py

Cài interface bằng UTF-8/JSON. Mọi thao tác file của PoC tập trung ở đây.

Hàm / lớp	Nhiệm vụ, đầu vào và đầu ra
JsonEvaluationRepository.readTextFile (filePath)	Đọc tối đa 1 MB + 1 byte để phát hiện file quá lớn; giải mã utf-8-sig. Không biến đổi xuống dòng. File quá lớn báo OSError, sai mã hóa báo UnicodeError.
JsonEvaluationRepository.__init__(reportDirectory)	Giữ thư mục mặc định cho kết quả khi không có --output.
loadInput(inputPath)	Gọi readTextFile rồi EvaluationInput.model_validate_json.
loadAnswer(answerPath)	Gọi readTextFile và trả nguyên văn; không gọi AI.
loadRecording(recordingPath)	Gọi readTextFile rồi RecordedReview.model_validate_json.
resolveOutput(outputPath)	Dùng đường dẫn chỉ định hoặc tạo report-UUID.json; từ chối file đã tồn tại trước khi chấm.
saveReport(report, outputPath)	Ghi JSON vào file tạm cùng thư mục, flush/fsync, tạo file đích bằng hard link và dọn file tạm. Nếu đích đã xuất hiện thì thất bại, không ghi đè. Cơ chế này cần filesystem hỗ trợ hard link.

7. Các Model dùng chung có trước PoC

Model/ids.py: các NewType như QuestionId, RubricId, CriterionId, AnswerId và VersionedRef định danh bản ghi. PoC tái sử dụng các ID này.

Model/question.py: ReviewStatus, LicenseStatus, ReviewChecks, ReviewMetadata, SourceMetadata, Skill, CriterionLevel, Criterion, Rubric và Question mô tả ngân hàng câu hỏi theo hợp đồng P03.

Model/evaluation.py: EvaluationStatus, CriterionStatus, AnswerRevisionRef, Evidence, CriterionCheck và EvaluationResult mô tả kết quả theo P03. Các lớp là dataclass/enum, không có thuật toán chấm riêng.

P03 chấm theo criterion và tham chiếu một rubric/câu; PoC theo abstraction mới có nhiều rubric theo năng lực và mức điểm theo rubric. Vì ý nghĩa trường khác nhau, gộp hai schema bằng cách đổi tên sẽ làm sai hợp đồng dùng chung. Quyết định này được ghi trong docs/data_contract.md; cần adapter hoặc thống nhất hợp đồng khi tích hợp với tính năng khác.

8. File kiểm tra: nhiệm vụ của các hàm

Phần này chỉ giải thích mã kiểm tra; không chứa số liệu hoặc kết quả chạy thử.

test/answerEvaluationTest.py

Kiểm tra hành vi chấm và bảo toàn căn cứ.

Hàm / lớp	Nhiệm vụ, đầu vào và đầu ra
inputs; recording; startEvaluation; statusCheck	Tạo dữ liệu và trạng thái hỗ trợ kiểm tra; không là một luồng đánh giá của ứng dụng.
testCompleteFlow; testLifecycleGuards	Kiểm tra chuỗi ADT, liên kết feedback và các chuyển trạng thái hợp lệ.
testScoreCombinations; testCustomScoreValues; testCountThresholds	Kiểm tra các khả năng met/unmet/chưa rõ, mức điểm tùy chỉnh và ngưỡng số tiêu chí.
testInvalidRubrics; testInvalidInterpretations; testQuoteOffsets	Kiểm tra từ chối rubric/ID/bằng chứng sai, vị trí trích dẫn và trích dẫn lặp.
testUnknownKeepsNull; testEmptyAnswer; testSchemaRejectsCoercion	Kiểm tra null, bài rỗng và từ chối ép kiểu sai.
testAtomicProviderFailure; testLateFailureAtomicity	Không giữ kết quả chấm dở dang khi nguồn diễn giải hoặc bước cuối lỗi.
testRecordingInputBinding; testRejectsChangedResults	Không dùng recording sai hash hoặc trạng thái bị sửa điểm không theo rubric.

test/evaluationArchitectureTest.py

Kiểm tra ranh giới lớp và khả năng thay thế phụ thuộc.

Hàm / lớp	Nhiệm vụ, đầu vào và đầu ra
testLayerDependencies; testNoFileAccess; testCompositionRoot	Kiểm tra import theo MVC-S, View/Controller không đọc file và main ghép đúng các phụ thuộc.
testControllerRejectsShape; testProgrammingErrorStaysVisible	Request sai bị chặn; lỗi lập trình không bị đổi thành lỗi file.
MemoryRepository	Cài năm thao tác Repository bằng dữ liệu mẫu và danh sách lưu trong bộ nhớ; dùng để quan sát Service độc lập với file.
testRepositoryCanBeReplaced; testFailureDoesNotSave; testNewAnswerRevision	Thay kho lưu, không lưu khi lỗi và không sửa câu trả lời cũ khi nhận bài mới.
testInjectedRulesAreUsed; testEvaluationsKeepSeparateState; testFactorySelectsProviders	Kiểm tra sử dụng đúng đối tượng rules, độc lập giữa các lượt chấm và chọn provider đúng cấu hình.
testConfigHidesSecret; testSingleRootEntryAndEnglishCode; testProductionLogicUsesObjects	API key không nằm trong repr; chỉ main và package marker ở cấp source; code source dùng tiếng Anh; nghiệp vụ là phương thức, chỉ main là hàm khởi chạy.

test/evaluationCommandTest.py

Kiểm tra đầu vào/đầu ra thực tế của lệnh chạy.

Hàm / lớp	Nhiệm vụ, đầu vào và đầu ra
tempPath; runCli	Tạo nơi ghi tạm và gọi chương trình CLI bằng Python hiện tại.
testCommandRoundTrip; testTextPresentation	Kiểm tra JSON in/lưu khớp nhau, không ghi đè và text có metadata/bằng chứng.
testNewAnswerNeedsReview; testInvalidFileStopsBeforeAssessment	Bài mới cần recording mới; sai UTF-8 hoặc quá dung lượng phải dừng trước khi tạo kết quả.

test/geminiInterpreterTest.py

Kiểm tra giao tiếp Gemini bằng phản hồi giả lập, không gọi mạng trong bộ kiểm tra.

Hàm / lớp	Nhiệm vụ, đầu vào và đầu ra
sampleInputs; sampleResponse; testStructuredRequest	Tạo dữ liệu giả lập, kiểm tra request có schema, đúng bài gốc và API key không nằm trong URL/body.
testHttpFailures; testConnectionFailures	Phân biệt quota, HTTP, timeout và lỗi mạng; không đưa dữ liệu lỗi bí mật ra thông báo.
testInvalidResponse; testRejectsProviderScore	Từ chối đầu ra dở dang/sai schema hoặc AI tự thêm điểm.
testPromptCanBeReplaced	Kiểm tra provider dùng đúng đối tượng prompt được truyền vào và lưu đúng phiên bản prompt.
testMissingConfiguration; testOversizeResponse	Từ chối thiếu key/model hoặc phản hồi quá lớn.

test/conftest.py; test/test_environment.py

Hạ tầng smoke test web của repo, không tham gia đánh giá câu trả lời.

Hàm / lớp	Nhiệm vụ, đầu vào và đầu ra
QuestionPreview; smoke_client; preview	Tạo FastAPI nhỏ, schema request và route /preview để kiểm tra thư viện web và template.
testWebStackRendersUnicode	Kiểm tra nội dung có ký hiệu Unicode được giữ đúng khi render.
test_web_stack_rejects_missing_question	Kiểm tra request thiếu question trả lỗi validation.

9. Hướng mở rộng và giới hạn thực tế

Thêm nguồn AI: viết lớp có interpret(inputs) và ba thuộc tính metadata theo EvidenceInterpreter, bổ sung lựa chọn trong InterpreterFactory và cập nhật tập provider được phép tại CLI/request/Service. Không cần đổi RubricRules hay FeedbackBuilder.

Đổi kho lưu: cài đủ năm thao tác EvaluationRepository và truyền instance mới tại Application. Giữ nguyên yêu cầu không ghi đè và chỉ lưu báo cáo hoàn chỉnh. Khi chuyển sang API cần thiết kế lại phần đường dẫn/request theo hợp đồng chung của nhóm.

Đổi chính sách điểm: thay/cải tiến RubricRules và truyền vào Service, đồng thời cập nhật schema khi cần. Việc thay class không tự bảo đảm chính sách mới còn đáp ứng bất biến; phải kiểm tra lại các trường hợp chưa đủ căn cứ và mức điểm.

Giới hạn: chưa thực thi code người dùng, chưa hỏi lại để làm rõ, chưa tổng hợp năng lực qua nhiều câu, chưa triển khai giao diện web. Dữ liệu ví dụ là nội dung minh họa chưa được duyệt độc lập. Diễn giải recorded không dùng để khẳng định chất lượng AI; lỗi provider không chứng minh câu trả lời sai.

Không đổi hợp đồng ngoài: các cờ CLI, field JSON và schemaVersion answerEvaluation/1 giữ nguyên; recording cũ vẫn dùng cùng inputDigest. Thay đổi nằm ở API Python nội bộ: gọi phương thức của đối tượng thay cho helper module-level. Chỉ một EvaluationService điều phối, một RubricRules tính mức, một EvidenceReview xác minh và một FeedbackBuilder tạo phản hồi.